Literature Review

Self-Supervised AI for ETL Error Detection and Root-Cause Localization

Akshat Sparsh (23BDS0149) and Anuprakash Dubey (23BDS0162)

October 2026

- Scope
- Create a Literature Review Matrix
- Identify Common Themes
- Compare Instead of Summarizing
- Find the Research Gap
- How This Project Addresses Each Gap
- Positioning

> Every reference below is a real paper, but confirm the venue, year and author list on Google Scholar or DBLP before submitting. A wrong year is an easy thing to be caught on and an easy thing to prevent. Drop anything you cannot verify.

This review follows the four steps set out in the guidance: build a review matrix, identify common themes, compare instead of summarising, and then find the research gap.

Scope

The project title sits where three research communities meet, and they mostly do not read one another:

1. Multivariate time-series anomaly detection, which is where the deep learning is.
2. Data quality and validation for pipelines, which is where the problem is.
3. Root-cause analysis for distributed systems, which is where the localization ideas are.

Papers were gathered from the major venues of each community (KDD, VLDB, SIGMOD, ICLR, AAAI, ICDM, CCS, ASE, NeurIPS) using combinations of the terms *self-supervised anomaly detection*, *multivariate time series*, *data quality validation*, *data pipeline monitoring*, *log anomaly detection* and

root cause localization. Work that is univariate only, or that assumes a fully labelled training set, was excluded, since neither assumption survives contact with a real ETL pipeline.

Create a Literature Review Matrix

No.	Reference	Problem Addressed	Dataset	Methodology	Results	Limitations	Key Contribution
1	Malhotra et al., ESANN 2015	Time-series anomaly detection	ECG, power demand	Stacked LSTM predictor, Gaussian error model	Detects anomalies without labels	Univariate focus, no localization	Prediction error is a usable anomaly signal
2	Malhotra et al., ICML AD Workshop 2016	Multi-sensor anomaly detection	Engine, power	LSTM encoder-decoder reconstruction	Works on unpredictable signals	Needs clean training data	Reconstruction beats prediction on noisy signals
3	Ruff et al., ICML 2018	One-class deep learning	MNIST, CIFAR	Deep SVDD, minimum-volume hypersphere	Strong on image one-class tasks	Prone to representation collapse	Made one-class objectives work end to end
4	Hundman et al., KDD 2018	Spacecraft telemetry anomalies	SMAP, MSL	LSTM prediction, nonparametric thresholding	Deployed at NASA JPL	Thresholding is heuristic	Released two widely used benchmarks
5	Su et al., KDD 2019	Multivariate server anomalies	SMD	Stochastic RNN with planar normalising flows	F1 about 0.86 on SMD	Slow, needs clean data	Probabilistic account of normality
6	Audibert et al., KDD 2020	Fast unsupervised detection	SMD, SWaT, SMAP	Adversarially trained autoencoder pair (USAD)	Comparable accuracy, far faster	Adversarial training unstable	Accuracy at production scale
7	Zhao et al., ICDM 2020	Correlations between sensors	SMAP, MSL, SMD	Graph attention over features and time	Beats non-graph baselines	Graph is learned, not given	Inter-variable structure carries signal
8	Deng and Hooi, AAAI 2021	Sensor anomalies with structure	SWaT, WADI	Learned sensor graph plus GNN forecasting	State of the art on water testbeds	Deviation ranking, no causal claim	Graph deviation scoring for localization
9	Tuli et al., VLDB 2022	Efficient transformer detection	Multiple public sets	Transformer, adversarial two-phase training	Faster training, competitive scores	Evaluated with point adjustment	Transformers for this problem
10	Xu et al., ICLR 2022	Separating anomalies from normal points	SMD, SWaT, PSM	Association discrepancy between attentions	Very high reported F1	Scores rest on point adjustment	Criterion beyond reconstruction error
11	Zerveas et al., KDD 2021	Representation learning for time series	UEA archive	Masked-value transformer pretraining	Beats supervised in low-label settings	Not built for anomaly detection	Masked modelling transfers to time series
12	Yue et al., AAAI 2022	Universal time-series representations	128 UCR sets	Hierarchical contrastive learning	Strong across many tasks	Contrastive views need care	Contrastive pretraining for time series
13	He et al., CVPR 2022	Scalable self-supervised vision	ImageNet	High-ratio masked autoencoding	87.8% ImageNet top-1	Vision domain	Aggressive masking makes the task real
14	Du et al., CCS 2017	Anomaly detection from system logs	HDFS, OpenStack	LSTM over log key sequences	F1 about 0.96 on HDFS	Needs stable log templates	First strong deep model for log anomalies
15	Meng et al., IJCAI 2019	Unstable log templates	HDFS, BGL	Template2Vec plus LSTM	Handles new templates	Still sequence-only	Log detection robust to drift
16	Guo et al., IJCNN 2021	Self-supervised log detection	HDFS, BGL, Thunderbird	BERT-style masked log-key modelling	Beats DeepLog	Logs only, no data content	Masked modelling on operational data
17	Schelter et al., VLDB 2018	Declarative data quality at scale	Amazon production data	Constraint checks, incremental metrics	Runs at production volume	Constraints written by hand	Data quality as a first-class system
18	Breck et al., SysML 2019	Data errors in ML pipelines	Google production	Schema inference, skew and drift checks	Caught real production bugs	Rule based, no learned model	Named the ETL error taxonomy

No.	Reference	Problem Addressed	Dataset	Methodology	Results	Limitations	Key Contribution
19	Song and He, SIGMOD 2021	Removing manual rule writing	7000 real columns	Mining data-domain patterns	Far fewer false positives	Single-column scope	Automated validation-rule discovery
20	Redyuk et al., EDBT 2021	Quality checks on changing data	Public and industry data	ML-based thresholds over quality metrics	Adapts to drift	Detection only, no root cause	Learned thresholds beat static ones
21	Wu et al., NOMS 2020	Root cause in microservices	Testbed traces	Attributed service graph, random walk	Localizes faulty service	Needs service call traces	Graph walking for localization
22	Li et al., KDD 2022	Root cause with interventions	Real service data	Causal graph, regression hypothesis testing	Beats correlation ranking	Needs a causal graph	Root cause as intervention recognition
23	Ikram et al., NeurIPS 2022	Root cause without a given graph	Microservice benchmarks	Hierarchical causal discovery	Scales to many metrics	Sensitive to independence tests	Causal discovery for failure localization
24	Sundararajan et al., ICML 2017	Attributing model outputs	Various	Integrated Gradients	Satisfies sensitivity, implementation invariance	Baseline choice matters	Axiomatic feature attribution
25	Lundberg and Lee, NeurIPS 2017	Unified explanation framework	Various	Shapley-value additive attribution	Consistent, model agnostic	Expensive to compute	Standard attribution tool
26	Kim et al., AAAI 2022	Are these benchmarks meaningful?	SMD, SWaT, SMAP, MSL	Re-evaluation under point adjustment	A random score beats published models	Critique, not a method	Showed the headline metric is broken
27	Wu and Keogh, TKDE 2023	Benchmark quality	Common AD benchmarks	Manual inspection of datasets	Triviality, mislabelling, run-to-failure bias	Critique, not a method	Argued for better benchmark design

Identify Common Themes

Reading across the matrix, the work falls into six groups.

Group A: Reconstruction and Prediction Methods

- Paper 1
- Paper 2
- Paper 5
- Paper 6

Group B: Structure-Aware and Graph-Based Methods

- Paper 7
- Paper 8

Group C: Transformer and Attention-Based Methods

- Paper 9
- Paper 10
- Paper 11

Group D: Self-Supervised Pretraining Methods

- Paper 11
- Paper 12
- Paper 13
- Paper 16

Group E: Data Quality and Pipeline Validation Systems

- Paper 17
- Paper 18
- Paper 19
- Paper 20

Group F: Root-Cause Localization and Explainable AI Methods

- Paper 21
- Paper 22
- Paper 23
- Paper 24
- Paper 25

Papers 26 and 27 sit outside these groups. They are critiques of how the field measures itself, and they shaped the evaluation protocol of this project more than any single method did.

This thematic grouping becomes the structure of the Related Work section.

Compare Instead of Summarizing

Learning what normal looks like (Group A)

The reconstruction line of work rests on a single assumption: train on normal data, and treat whatever the model reconstructs badly as abnormal. Malhotra et al. established that prediction error alone carries the signal, then found that reconstruction holds up better when the series is noisy and genuinely unpredictable, which is the more common case. OmniAnomaly extended this by making normality probabilistic rather than a point estimate, so the model can express how confident it is. That gains accuracy at a real cost in speed, which is exactly what USAD then attacked, using two adversarially trained autoencoders to reach similar accuracy far faster.

What none of them question is the training data. All four assume a clean, or near-clean, set of normal examples. In an ETL setting that assumption quietly fails, because nobody knows which historical runs were healthy. The incidents that were never noticed are still sitting in the history, and they are precisely the ones a detector would need to learn to distrust.

Adding structure, and stopping short of causality (Groups B and F)

Group A treats variables as an unordered bundle. MTAD-GAT and GDN both push back on that, and both improve on the flat baselines, which is good evidence that inter-variable structure carries real signal. The difference between them is where the graph comes from. MTAD-GAT learns attention over features, while GDN learns an explicit sensor graph and then scores each node by how far it deviates from what its neighbours predict.

GDN comes closest to what this project needs, and the gap between the two is instructive. GDN's deviation score ranks sensors by anomaly magnitude, and its authors describe this as localization. In a sensor network that is defensible. In a pipeline it is not, because magnitude does not survive propagation. A five percent null rate at extraction becomes a thirty percent revenue drop three tasks later, so the largest deviation lands on the task furthest from the fault. Any ranking by magnitude will systematically blame the wrong end of the graph.

Groups B and F therefore fail to connect. The structure-aware detectors have the graph but rank by magnitude, and the root-cause work reasons about causality but operates on service call traces rather than on data content.

Self-supervision, borrowed but not yet arrived (Group D)

Group D is the newest thread and the most promising. Zerveas et al. showed masked modelling helps on time series, TS2Vec showed contrastive learning does too, and LogBERT showed masked pretraining transfers to operational data specifically. MAE supplied the design lesson that matters most here, which is that masking has to be aggressive enough that the task cannot be solved by local interpolation.

None of this has reached ETL data quality. LogBERT is the nearest neighbour, and it models log key sequences, so it sees that a job ran and in what order but never what the data looked like. A pipeline can

execute flawlessly, emit a perfect log, and load a table where every amount is off by a factor of a hundred.

The systems community has the problem but not the model (Group E)

Group E is where the actual problem lives. Deequ made data quality checks a production concern, TFDV built the schema and skew machinery that much of the industry now copies, and both encode a taxonomy of ETL failures more grounded than anything in the machine learning papers. The limitation they share is that somebody has to write the rules. Auto-Validate is the most serious attempt to remove that burden and it works well, but it stays within a single column, so it cannot see a fault that only appears as a relationship between two tables. Redyuk et al. learn thresholds instead of fixing them, which handles drift, but the output is still a flag on a metric rather than a diagnosis.

How the field grades itself (Papers 26 and 27)

These two deserve more attention than their citation counts suggest. Kim et al. re-evaluated the standard benchmarks under point-adjusted F1, the metric almost every paper in Groups A and C reports, and found that a randomly generated score can beat published models. The reason is mechanical: point adjustment credits you with an entire anomalous segment for detecting any single timestep inside it, so on datasets with long anomalous stretches, alerting at random works. Wu and Keogh reached a related conclusion from the data side, documenting mislabelling and triviality in the benchmarks themselves.

Taken together, a large fraction of the reported progress in Groups A and C may be an artefact of measurement. That is not a reason to discard the methods, but it is a strong reason not to report point-adjusted F1 as a headline number, and to be suspicious of any comparison that exists only in that metric.

Find the Research Gap

Dataset Gap

- **No benchmark with root-cause labels?** Yes. There is no public benchmark of ETL pipeline failures with root-cause annotations.
- **Wrong kind of data?** Yes. Groups A, B and C evaluate almost entirely on SMD, SWaT, SMAP and MSL, which are server and sensor telemetry. None has a task graph, none has data-content quality metrics, and none labels which component originated a failure.
- **Lack of real-world validation?** Partly the opposite. Group E works on real pipeline data, but that data is proprietary and unreleased, so nobody else can build on it.

The practical result is that root-cause localization in a data pipeline cannot currently be posed as a benchmark task, let alone measured. Wu and Keogh's criticism applies with extra force here: the field is not merely using flawed benchmarks for this problem, it has none at all.

Methodological Gap

- **Detection and localization treated separately?** Yes. Groups A to D detect, Group F localizes, and the localization work needs service call traces that a data pipeline does not produce.
- **Localization by magnitude only?** Yes. Where localization does appear inside a detector it is deviation ranking, which propagation defeats for the reason given above.
- **No self-supervision on data quality?** Yes. Group D has the machinery and Group E has the problem, and nobody has connected them.
- **No use of pipeline structure?** Mostly. Graph methods exist for sensors, but the DAG of an ETL pipeline is known in advance and goes unused.

Explainability Gap

- **Black-box predictions?** Largely. Explanation in Groups A to C means a reconstruction-error bar chart per variable, which says which number moved rather than what kind of failure occurred.
- **No explainable AI?** Integrated Gradients and SHAP are mature and rarely applied here.
- **Explanations never checked?** Yes, and this is the sharper gap. When attribution is shown it is almost never scored, because there is no ground truth to score it against.

Application Gap

- **Binary classification only?** Yes. Nearly all of this work outputs a single flag.
- **No fault typing?** Yes. Knowing a run is broken is different from knowing it is a schema drift rather than a unit change.
- **Nothing an engineer can act on?** An on-call engineer at 3am needs three things: whether the run is broken, which task broke it, and what kind of break it is. No single system in the matrix produces all three.
- **No handling of benign change?** Yes. Real pipelines have seasonality, growth and one-off events such as marketing campaigns, all of which move volume sharply and none of which is a fault. The public benchmarks contain little of this, so published false-positive rates are optimistic.

Evaluation Gap

This category is additional to the four in the guidance, and Papers 26 and 27 force it.

- **Inflated metrics?** Point-adjusted F1 is the field standard and is known to be broken to the point where random noise beats real models.
- **Unrealistic training assumptions?** Training sets are assumed clean when real history is not.
- **Leakage through splits?** Random rather than chronological splits let information from the future reach the past.

How This Project Addresses Each Gap

Gap	Response
Dataset	A nine-task warehouse pipeline simulated at row level with fifteen fault types, so propagation through the DAG is emergent rather than scripted. Every run is labelled with its origin task, its measured blast radius and its fault type.
Methodological	One self-supervised model that detects and localizes together. Localization uses an ancestor-conditioned score rather than deviation ranking: a task whose output follows from its inputs is a victim, and a task whose output is surprising given healthy inputs is the origin.
Explainability	Attribution over eight named symptom families rather than raw feature indices, and because the simulator knows which family each fault should move, attribution accuracy is a number rather than a picture.
Application	Three outputs per run: is it broken, which task, and which symptom. Seasonality, growth, drift and campaigns are generated and labelled normal on purpose, so false positives on ordinary business variation are visible in the results.
Evaluation	AUPRC and F1 at a fixed alert budget as headline metrics, point-adjusted F1 reported only alongside and labelled, three percent contamination left in the training history, chronological splits, and normalisation fitted on the training split alone.

Positioning

The closest neighbours to this work are GDN, which brings graph structure to anomaly detection but ranks by deviation magnitude, LogBERT, which brings masked self-supervision to operational data but models log sequences rather than data content, and TFDV, which has the right taxonomy of ETL failures but detects them with hand-written rules. This project occupies the space those three leave open: self-supervised learning over data-quality profiles, structured by the pipeline DAG, producing a root-cause score designed to survive propagation, and evaluated on a benchmark built specifically because none existed.